

Functions, Class and Objects

Prof. Mahipal Khoja

Assistant Professor

Artificial Intelligence and Data Science Department

Content

1. Concept of Functions in C++.....	3
2. C++ User Defined Functions.....	5
3. Function Calling.....	7
4 . Function Parameters.....	8
4 . Function Arguments.....	11
5. Default and Constant Arguments.....	13
6. Return Statement.....	18
6. Function Prototyping.....	22
7. C++ Library Functions.....	25
7. Function Overloading.....	27
8. Inline Functions.....	31
8. Defining Class and Creating Objects.....	34
9. Defining Member function.....	43
10. Nesting of Member functions.....	47
11. Private Member functions.....	56
12. Friend function.....	58

Concept of Functions in C++

- A function is a group of statements that together perform a task. Every C++ program has at least one function, which is **main()**, and all the most trivial programs can define additional functions.
- You can divide up your code into separate functions. How you divide up your code among different functions is up to you, but logically the division usually is such that each function performs a specific task.
- A function declaration tells the compiler about a function's name, return type, and parameters. A function definition provides the actual body of the function.

Concept of Functions in C++

- Suppose we need to create a program to create a circle and color it. We can create two functions to solve this problem:
 - a function to draw the circle
 - a function to color the circle
- Dividing a complex problem into smaller chunks makes our program easy to understand and reusable.
- There are two types of function:
 - Standard Library Functions: Predefined in C++
 - User-defined Function: Created by users

C++ User Defined Functions

- C++ allows the programmer to define their own function.
- A user-defined function groups code to perform a specific task and that group of code is given a name (identifier).
- When the function is invoked from any part of the program, it all executes the codes defined in the body of the function.

C++ Function Declaration

- The syntax to declare a function is:

```
returnType functionName (parameter1, parameter2,...) {  
    // function body  
}
```

C++ User Defined Functions

- Here's an example of a function declaration.

```
// function declaration
```

```
void greet() {  
    cout << "Hello World";  
}
```

Here,

- the name of the function is **greet()**
- the return type of the function is **void**
- the empty parentheses mean it doesn't have any **parameters**
- the function body is written **inside {}**

Calling a Function

In the above program, we have declared a function named `greet()`. To use the `greet()` function, we need to call it. Here's how we can call the above `greet()` function.

```
int main() {
```

```
    // calling a function  
    greet();
```

```
}
```

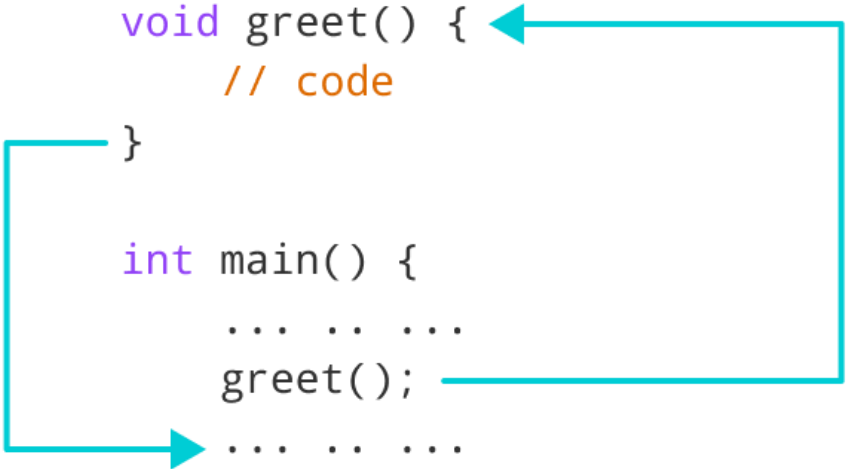
```
#include<iostream>
```

```
void greet() {  
    // code
```

```
}
```

```
int main() {  
    ... ..  
    greet();  
    ... ..  
}
```

**function
call**



The diagram illustrates the function call process. A blue line starts from the `greet();` line in the `main()` function, goes to the right, then up, then left, and finally down to the opening curly brace of the `greet()` function definition. A blue arrowhead points to the opening curly brace of the `greet()` function, indicating the point of execution transfer.

Function Parameters

a function can be declared with parameters (arguments). A parameter is a value that is passed when declaring a function. For example, let us consider the function below:

```
void printNum(int num) {  
    cout << num;  
}
```

Here, the int variable num is the function parameter. We pass a value to the function parameter while calling the function.

```
int main() {  
    int n = 7;  
    // calling the function  
    // n is passed to the function as argument  
    printNum(n);  
    return 0;  
}
```


Function Parameters

Example 2: Function with Parameters

```
// program to print a text
#include <iostream>
using namespace std;
// display a number
void displayNum(int n1, float n2) {
    cout << "The int number is " << n1;
    cout << "The double number is " << n2;
}
int main() {
    int num1 = 5;
    double num2 = 5.5;
    // calling the function
    displayNum(num1, num2);
    return 0;
}
```

Function Parameters

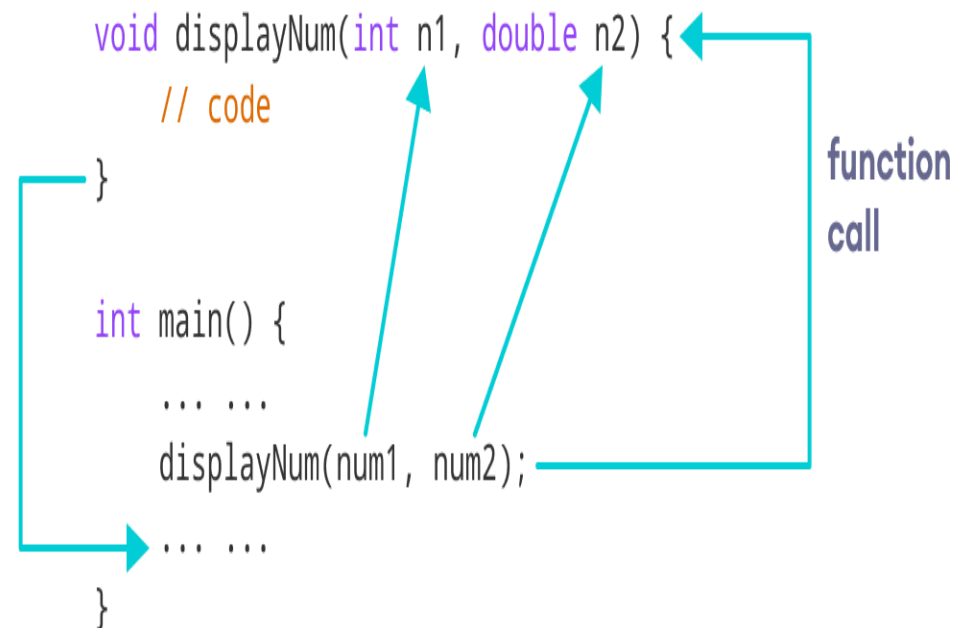
- In the above program, we have used a function that has one int parameter and one double parameter.
- We then pass num1 and num2 as arguments. These values are stored by the function parameters n1 and n2 respectively.

Note: The type of the arguments passed while calling the function must match with the corresponding parameters defined in the function declaration.

```
#include<iostream>

void displayNum(int n1, double n2) {
    // code
}

int main() {
    ... ..
    displayNum(num1, num2);
    ... ..
}
```



The diagram illustrates the function call process. A red arrow points from the `displayNum(num1, num2);` line in the `main()` function to the `displayNum(int n1, double n2) {` function declaration. Another red arrow points from the `displayNum(num1, num2);` line to the `int n1` parameter in the function declaration. A third red arrow points from the `displayNum(num1, num2);` line to the `double n2` parameter in the function declaration. The text "function call" is written next to the arrow pointing to the function declaration.

Function Arguments

If a function is to use arguments, it must declare variables that accept the values of the arguments. These variables are called the formal parameters of the function.

The formal parameters behave like other local variables inside the function and are created upon entry into the function and destroyed upon exit.

While calling a function, there are two ways that arguments can be passed to a function –

Call by Value

This method copies the actual value of an argument into the formal parameter of the function. In this case, changes made to the parameter inside the function have no effect on the argument.

Function Arguments

Call by Pointer

This method copies the address of an argument into the formal parameter. Inside the function, the address is used to access the actual argument used in the call. This means that changes made to the parameter

Call by Reference

This method copies the reference of an argument into the formal parameter. Inside the function, the reference is used to access the actual argument used in the call. This means that changes made to the parameter affect the argument.

By default, C++ uses call by value to pass arguments.

Default Arguments and Constant Arguments

- In C++ programming, we can provide default values for function parameters.
- If a function with default arguments is called without passing arguments, then the default parameters are used.
- However, if arguments are passed while calling the function, the default arguments are ignored.

Case 1 : No argument is passed

```
void temp(int = 10, float = 8.8);

int main() {
    ...
    temp();
    ...
}

void temp(int i, float f) {
    // code
}
```

Case 2 : First argument is passed

```
void temp(int = 10, float = 8.8);

int main() {
    ...
    temp(6);
    ...
}

void temp(int i, float f) {
    // code
}
```

Case 3 : All arguments are passed

```
void temp(int = 10, float = 8.8);

int main() {
    ...
    temp(6, -2.3);
    ...
}

void temp(int i, float f) {
    // code
}
```

Case 4 : Second argument is passed

```
void temp(int = 10, float = 8.8);

int main() {
    ...
    temp(3.4);
    ...
}

void temp(int i, float f) {
    // code
}
```

Default Arguments and Constant Arguments

- We can understand the working of default arguments from the image above:
- When `temp()` is called, both the default parameters are used by the function.
- When `temp(6)` is called, the first argument becomes 6 while the default value is used for the second parameter.
- When `temp(6, -2.3)` is called, both the default parameters are overridden, resulting in `i = 6` and `f = -2.3`.
- When `temp(3.4)` is passed, the function behaves in an undesired way because the second argument cannot be passed without passing the first argument.
- Therefore, 3.4 is passed as the first argument. Since the first argument has been defined as `int`, the value that is actually passed is 3.

Default Arguments and Constant Arguments

- Constant arguments in C++ are function parameters declared with the `const` keyword, ensuring that their values cannot be altered within the function. This provides a safeguard against accidental changes to the arguments, which can be crucial in certain situations where data integrity is a priority.

```
void functionName(const int x)  
{  
    // x cannot be modified here  
}
```

In the above example, `x` is a constant argument. Any attempt to modify `x` within the function will result in a compiler error.

Features of Constant Arguments in C++

- **Immutability:** Once a constant argument is passed to a function, its value cannot be modified. This behavior is similar to a constant variable in C++.
- **Flexibility in Parameter List:** Any parameter in a function's parameter list can be declared as a constant argument, regardless of its position.
- **Overloading Constraints:** Functions cannot be overloaded solely based on the presence of constant arguments. For example, the functions `void func(int x)` and `void func(const int x)` would cause a conflict because the compiler does not differentiate between constant and non-constant integer arguments for overloading purposes.
- **Default Values:** Constant arguments can have default values, but like other default arguments, these values can be overridden when the function is called. However, once assigned, the value cannot be modified within the function.

Difference between Default and Constant Arguments in C++

Aspect	Default Arguments	Constant Arguments
Purpose	Provides default values for function parameters.	Prevents modification of the argument's value.
Declaration	Declared with a value in the function declaration.	Declared with the <code>const</code> keyword.
Modifiability	Can be modified within the function if not declared <code>const</code> .	Cannot be modified within the function.
Overloading	Allows overloading if function signatures differ.	Cannot be used alone to differentiate overloaded functions.
Value Assignment	Automatically assigned if no value is passed during function call.	Value is passed by the caller and cannot be changed.
Behavior in Function	Acts as a normal parameter unless overridden by the caller.	Acts as a read-only parameter within the function.

Return Statement

- In the above programs, we have used void in the function declaration. For example,

```
void displayNumber() {  
    // code  
}
```

- This means the function is not returning any value.
- It's also possible to return a value from a function. For this, we need to specify the returnType of the function during function declaration.
- Then, the return statement can be used to return a value from a function.

Return Statement

For example,

```
int add (int a, int b) {  
    return (a + b);  
}
```

- Here, we have the data type int instead of void. This means that the function returns an int value.
- The code return (a + b); returns the sum of the two parameters as the function value.
- The return statement denotes that the function has ended. Any code after return inside the function is not executed.

Return Statement

Example 3: Add Two Numbers

```
#include <iostream>
using namespace std;
// declaring a function
int add(int a, int b) {
    return (a + b);
}
int main() {
    int sum;
    // calling the function and storing
    // the returned value in sum
    sum = add(100, 78);
    cout << "100 + 78 = " << sum << endl;
    return 0;
}
```

Return Statement

- In the above program, the add() function is used to find the sum of two numbers.
- We pass two int literals 100 and 78 while calling the function.
- We store the returned value of the function in the variable sum, and then we print it.

Notice that sum is a variable of int type. This is because the return value of add() is of int type.

```
#include<iostream>

int add(int a, int b) {
    return (a + b);
}

int main() {
    int sum;
    sum = add(100, 78);
    ... ..
}
```

function call

The diagram illustrates the flow of data between the `main` function and the `add` function. A blue arrow originates from the `add(100, 78);` line in the `main` function and points to the `return (a + b);` line in the `add` function, representing the return value being passed back. Another blue arrow originates from the `return (a + b);` line and points to the `sum` variable in the `main` function, indicating that the returned value is stored in `sum`. A third blue arrow points from the `sum` variable to the `...` line, indicating that the value is then printed or used further.

Function Prototype

- In C++, the code of function declaration should be before the function call. However, if we want to define a function after the function call, we need to use the function prototype. For example,
- In the below code, the function prototype is:
void add(int, int);

```
// function prototype
void add(int, int);

int main() {
    // calling the function before declaration.
    add(5, 3);
    return 0;
}

// function definition
void add(int a, int b) {
    cout << (a + b);
}
```

Function Prototype

- This provides the compiler with information about the function name and its parameters. That's why we can use the code to call a function before the function has been defined.
- The syntax of a function prototype is:

returnType functionName(dataType1, dataType2, ...);

Benefits of Using User-Defined Functions

- Functions make the code reusable. We can declare them once and use them multiple times.
- Functions make the program easier as each small task is divided into a function.
- Functions increase readability.

C++ Library Functions

- Library functions are the built-in functions in C++ programming.
- Programmers can use library functions by invoking the functions directly; they don't need to write the functions themselves.
- Some common library functions in C++ are `sqrt()`, `abs()`, `isdigit()`, etc.
- In order to use library functions, we usually need to include the header file in which these library functions are defined.
- For instance, in order to use mathematical functions such as `sqrt()` and `abs()`, we need to include the header file `cmath`.

C++ Library Functions

C++ Program to Find the Square Root of a Number

In this program, the `sqrt()` library function is used to calculate the square root of a number.

The function declaration of `sqrt()` is defined in the `cmath` header file. That's why we need to use the code `#include <cmath>` to use the `sqrt()` function.

```
#include <iostream>
#include <cmath>
using namespace std;

int main() {
    double number, squareRoot;

    number = 25.0;

    // sqrt() is a library function to calculate the square root
    squareRoot = sqrt(number);

    cout << "Square root of " << number << " = " << squareRoot;

    return 0;
}
```

Function Overloading

- In C++, two functions can have the same name if the number and/or type of arguments passed is different.
- These functions having the same name but different arguments are known as overloaded functions. For example:

```
// same name different arguments
int test() { }
int test(int a) { }
float test(double a) { }
int test(int a, double b) { }
```

- Here, all 4 functions are overloaded functions.
- Notice that the return types of all these 4 functions are not the same. Overloaded functions may or may not have different return types but they must have different arguments

Working of Function Overloading

In this program, we overload the `absolute()` function. Based on the type of parameter passed during the function call, the corresponding function is called.

```
float absolute(float var) {  
    // code  
}
```

```
int absolute(int var) {  
    // code  
}
```

```
int main() {
```

```
    absolute(-5);
```

```
    absolute(5.5f);
```

```
    ... ..
```

```
}
```



Working of Function Overloading

```
void display(int var1, double var2) {  
    // code  
}  
  
void display(double var) {  
    // code  
}  
  
void display(int var) {  
    // code  
}  
  
int main() {  
    int a = 5;  
    double b = 5.5;  
  
    display(a);  
    display(b);  
    display(a, b);  
  
    ... ..  
}
```

The diagram illustrates function overloading with three overloaded `display` functions and a `main` function. The `main` function calls `display(a)`, `display(b)`, and `display(a, b)`. Red arrows show the following mappings:

- `display(a)` calls the `display(int var)` function.
- `display(b)` calls the `display(double var)` function.
- `display(a, b)` calls the `display(int var1, double var2)` function.

Working of Function Overloading

- Here, the display() function is called three times with different arguments. Depending on the number and type of arguments passed, the corresponding display() function is called.
- The return type of all these functions is the same but that need not be the case for function overloading.

Inline Function

- An inline function is a function in C++ whose code is expanded at the point of call at compile time. It reduces function-call overhead.
- The inline keyword suggests replacing a function call with its code to reduce overhead.
- Inlining is a request, not guaranteed by the compiler.
- The compiler may ignore inlining if the function contains loops, recursion, static variables, switch/goto, or a non-void function without a return statement..

Inline Function

```
inline void displayNum(int num) {  
    cout << num << endl;  
}  
  
int main() {  
    displayNum(5);  
    displayNum(8);  
    displayNum(666);  
}
```

Compilation →

```
inline void displayNum(int num) {  
    cout << num << endl;  
}  
  
int main() {  
    cout <<< 5 << endl;  
    cout << 8 << endl;  
    cout << 666 << endl;  
}
```


Inline Function

Need for Inline Functions

- Function calls involve overhead from storing the return address, passing arguments, and returning control, which can be significant for small, frequently used functions.
- Inline functions help eliminate this overhead by replacing the function call with the actual code, improving efficiency.
- Inline functions are useful only when the function call overhead is higher than the function's execution time.

```
#include <stdio.h>

// Inline function (but compiler may ignore it)
inline void displayMessage()
{
    for (int i = 0; i < 5; i++)
    {
        printf("Hello %d\n", i);
    }
}

int main()
{
    displayMessage();
    return 0;
}
```

Defining Class and Creating Objects

- Objects and classes are used to wrap related functions and data in one place in C++.
- Suppose we need to store the length, breadth, and height of a rectangular room and calculate its area and volume.
- To handle this task, we can create three variables, say, length, breadth, and height, along with the functions `calculate_area()` and `calculate_volume()`.
- However, in C++, rather than creating separate variables and functions, we can also wrap the related data and functions in a single place (by creating objects).
- This programming paradigm is known as object-oriented programming.
- But before we can create objects and use them in C++, we first need to learn about classes.

Defining Class and Creating Objects

C++ Class

- A class is a blueprint for the object.
- We can think of a class as a sketch (prototype) of a house.
- It contains all the details about the floors, doors, windows, etc - we build the house based on these descriptions.
- The house is the object.

Defining Class and Creating Objects

Create a Class

- A class is defined in C++ using the keyword `class` followed by the name of the class.
- The body of the class is defined inside curly brackets and terminated by a semicolon at the end.

```
class ClassName {  
    // some data  
    // some functions  
};
```

Defining Class and Creating Objects

Create a Class

```
class Room {  
    public:  
        double length;  
        double breadth;  
        double height;  
  
        double calculate_area(){  
            return length * breadth;  
        }  
  
        double calculate_volume(){  
            return length * breadth * height;  
        }  
};
```

Defining Class and Creating Objects

Create a Class

- Here, we defined a class named Room.
- The variables length, breadth, and height declared inside the class are known as data members.
- And the functions calculate_area() and calculate_volume () are known as member functions of a class.

Defining Class and Creating Objects

C++ Objects

When a class is defined, only the specification for the object is defined; no memory or storage is allocated.

To use the data and access functions defined in the class, we need to create objects.

Defining Class and Creating Objects

C++ Objects

Syntax to Define Object in C++

ClassName object_name;

We can create objects of Room class (defined in the above example) as follows:

```
// sample function
void sample_function() {
    // create objects
    Room room1, room2;
}

int main(){
    // create objects
    Room room3, room4;
}
```


Defining Class and Creating Objects

C++ Objects

- Here, two objects room1 and room2 of the Room class are created in sample_function().
- Similarly, the objects room3 and room4 are created in main().
- As we can see, we can create objects of a class in any function of the program.
- We can also create objects of a class within the class itself or in other classes.
- Also, we can create as many objects as we want from a single class.

C++ Access Data Members and Member Functions

- We can access the data members and member functions of a class by using a . (dot) operator.
- For example,
room2.calculate_area();
- This will call the calculate_area() function inside the Room class for object room2.
- Similarly, the data members can be accessed as:
room1.length = 5.5;
- In this case, it initializes the length variable of room1 to 5.5.

Defining Member function

- We know that only the public members of a class can be accessed by the object of that class, using dot operator.
- However a member function can call another member function of the same class directly without using the dot operator. This is called as nesting of member functions.
- A member function can call another member function of the same class for that you do not need an object

Defining Member functions

Definition of member function outside the class

```
class book {  
    // body of the class  
};
```

```
void book :: getdata(char a[],float b) {  
    // defining member function outside the class  
    strcpy(title,a);  
    price = b;  
}
```

```
void book :: putdata () {  
    cout<<"\nTitle of Book: "<<title;  
    cout<<"\nPrice of Book: "<<price;  
}
```

Defining of Member functions

- Defining a member function outside a class requires the function declaration (function prototype) to be provided inside the class definition.
- The member function is declared inside the class like a normal function. This declaration informs the compiler that the function is a member of the class and that it has been defined outside the class.
- After a member function is declared inside the class, it must be defined (outside the class) in the program.
- *The definition of member function outside the class differs from normal function definition, as the function name in the function header is preceded by the class name and the scope resolution operator (: :)*

Note that the member functions of the class can access all the data members and other member functions of the same class (private, public or protected) directly by using their names. In addition, different classes can use the same function name.

Defining Member functions

Definition of member function inside the class

```
class book {  
    char title[30];  
    float price;  
    public:  
        void getdata(char [],float); // declaration  
        void putdata()//definition inside the class {  
            cout<<"\nTitle of Book: "<<title;  
            cout<<"\nPrice of Book: "<<price;  
        };  
};
```

- A member function of a class can also be defined inside the class. However, when a member function is defined inside the class, the class name and the scope resolution operator are not specified in the function header.
- Moreover, the member functions defined inside a class definition are by default inline functions.

Nesting of Member functions

- Nesting of member functions occurs when one member function calls another within the same class.
- A member function can be called using the dot (.) operator with an object or directly by other member functions of the same class.
- All member functions can access and call other member functions within the class.
- Static member functions can be called by other member functions but can only directly call other static member functions.
- Member functions can be recursive, meaning they can call themselves.
- Member functions defined outside the class can also exhibit nesting, adhering to accessibility and scope rules.

Nesting of Member functions

```
#include<iostream>
using namespace std;

class c1
{
    int s = 2;
public:
    void m1()
    {
        cout << "m1 ";
    }
    void m2()
    {
        // Non-static member function calling another non-static member function
        m1();
        cout << "m2 ";
    }
};

int main()
{
    c1 obj;
    obj.m2();
}
```


Nesting of Member functions

- In this example, `m2()` calls another member function `m1()` within the same class, demonstrating nesting of member functions. When `obj.m2()` is executed, it first prints “m1” from `m1()` and then “m2” from `m2()`, resulting in the output “m1 m2”.

Advantages of Nesting Member Functions in C++

- **Code Reusability:** Nesting allows you to reuse code within the same class. For example, if a calculation is needed in multiple functions, it can be encapsulated in a separate member function and then called where necessary.
- **Modularity:** By breaking down a complex task into smaller functions, the code becomes more modular. Each function handles a specific task, making the code easier to maintain and understand.
- **Encapsulation:** Nested functions can help keep implementation details hidden. For instance, the `calculateArea` function is private and can only be accessed within the class, ensuring that the internal workings are not exposed outside the class.

Nesting of Member functions

Static Function calling Non Static Function.

The given code gives an error because the member function m2() is which is static is trying to call non-static member function m1(). Static member functions don't have access to non-static members of their class.

```
#include<iostream>
using namespace std;

class c1
{
    int s=2;
public:
    void m1()
    {
        cout<<"m1";
    }
    void static m2()
    {
        m1();
        cout<<"m2";
    }
};

int main()
{
    c1 obj;
    obj.m2();
}
```

Nesting of Member functions

Indirect Recursion

The given code will give a segmentation fault because the member function `m1()` is calling `m2()` and `m2()` is calling `m1()`. This is a case of indirect recursion. However, since the base case is missing this will trap the code in an infinite loop.

```
#include<iostream>
using namespace std;

class c1
{
    public:
        void m1()
        {
            cout<<"m1";
            m2();
        }
        void m2()
        {
            m1();
            cout<<"m2";
        }
};

int main()
{
    c1 obj;
    obj.m2();
}
```

Nesting of Member functions

Private Member Function Access

Explanation: Member functions can even access and call private and protected member functions of their class.

```
#include<iostream>
using namespace std;

class c1{
    void m1()
    {
        cout<<"m1";
    }
public:
    void m2()
    {
        m1();
        cout<<"m2";
    }
};

int main()
{
    c1 obj;
    obj.m2();
}
```

Nesting of Member functions

Recursive Function Calls

Explanation: The given code will give a segmentation fault because the member function m2() is calling m1() and m1() is calling itself. This is a case of recursion. However, since the base case is missing this will trap the code in an infinite loop.

```
#include<iostream>
using namespace std;

class c1
{
    void m1()
    {
        cout<<"m1";
        m1();
    }
public:
    void m2()
    {
        m1();
        cout<<"m2";
    }
};

int main()
{
    c1 obj;
    obj.m2();
}
```

Nesting of Member functions

Recursive Parameter Handling

Explanation: This example demonstrates a recursive function that decrements its parameter and prints numbers in descending order.

```
#include<iostream>
using namespace std;

class c1
{
    public:
        void m1(int i)
        {
            if(i>0)
                m1(i-1);
            cout<<i;
        }
};

int main()
{
    c1 obj;
    obj.m1(5);
}
```

Nesting of Member functions

Hierarchical Nesting

Explanation: Nesting of functions can be done in a hierarchical manner where one member function calls another member function which in turn call another member function.

```
class c1
{
    public:
        void m1()
        {
            cout<<"m1";
            m2();
        }
        void m2()
        {
            m3();
            cout<<"m2";
        }
        void m3()
        {
            cout<<"m3";
        }
};

int main()
{
    c1 obj;
    obj.m1();
}
```

Private Member functions

- Although it is normal practice to place all the data items in private section and all the functions in public, some situations may require certain functions to be hidden (like private data) from the outside calls.
- Tasks such as deleting an account of a customer file, or providing an increment to any employee are events of serious consequences and therefore the functions handling such tasks should have restricted access.
- We can place these functions in the private scope.
- A private member function can only be called by another function that is a member of the same or its class.
- Even an object cannot invoke a private function using the `.(dot)` operator.
consider class as defined below

Private Member functions

```
class student
{
    int roll;
    void read(); // private member function
public:
    void update();
    void write();
};
```

- So if **S** is the object of **student** class then
- **S.read(); // will not work m object cannot access private members or member functions**
- It is illegal however the function read() can be called only by the function update() to update the value of m

Friend function

Data hiding is a fundamental concept of object-oriented programming. It restricts the access of private members from outside of the class.

Similarly, protected members can only be accessed by derived classes and are inaccessible from outside. For example,

```
class MyClass {  
    private:  
        int member1;  
}  
  
int main() {  
    MyClass obj;  
  
    // Error! Cannot access private members from here.  
    obj.member1 = 5;  
}
```

Friend function

- However, there is a feature in C++ called friend functions that break this rule and allow us to access member functions from outside the class.
- Similarly, there is a friend class as well, which we will learn later in this tutorial.
- A friend function can access the private and protected data of a class. We declare a friend function using the friend keyword inside the body of the class.

```
class className {  
    ... ..  
    friend returnType functionName(arguments);  
    ... ..  
}
```

Working of friend Function

```
#include <iostream>
using namespace std;

class Distance {
    private:
        int meter;

        // friend function
        friend int addFive(Distance);

    public:
        Distance() : meter(0) {}
};

// friend function definition
int addFive(Distance d) {

    //accessing private members from the friend function
    d.meter += 5;
    return d.meter;
}

int main() {
    Distance D;
    cout << "Distance: " << addFive(D);
    return 0;
}
```

Friend function

- Here, `addFive()` is a friend function that can access both private and public data members.
- Though this example gives us an idea about the concept of a friend function, it doesn't show any meaningful use.
- A more meaningful use would be operating on objects of two different classes. That's when the friend function can be very helpful..

friend Class in C++

- We can also use a friend Class in C++ using the friend keyword. For example,
class ClassB;

```
class ClassA {  
    // ClassB is a friend class of ClassA  
    friend class ClassB;  
    ... ..  
}
```

```
class ClassB {  
    ... ..  
}
```

friend Class in C++

- When a class is declared a friend class, all the member functions of the friend class become friend functions.
- Since ClassB is a friend class, we can access all members of ClassA from inside ClassB.
- However, we cannot access members of ClassB from inside ClassA. It is because friend relation in C++ is only granted, not taken.

Parul[®]
University

NAAC
GRADE **A++**



<https://paruluniversity.ac.in/>

